صفحات مشترک پیاده‌سازی می‌شوند. برای این که امکان اشتراک وجود داشته باشد بایستی کد آن چندمدخلی باشد. نباید طبیعت فقط‌خواندنی کد مشترک به حساب صحت کد گذاشته شود؛ بایستی سیستم‌عامل این ویژگی را اجبار کند.

اشتراک حافظه مابین پردازه‌های یک سیستم مشابه اشتراک فضای آدرس یک وظیفه توسط نخ‌های آن است که در فصل ۴ عنوان شد. به علاوه، از فصل ۳ به خاطر دارید که اشتراک حافظه را به عنوان روشی برای ارتباط میان پردازه‌ها مطرح کردیم. برخی سیستم‌عامل‌ها، اشتراک حافظه را با استفاده از صفحات مشترک پیاده‌سازی می‌کنند.

سازماندهی حافظه مطابق صفحات، علاوه بر امکان اشتراک صفحات فیزیکی یکسان میان چندین پردازه، منافع متعدد دیگری هم دارد. چندین نفع دیگر را در فصل ۱۰ پوشش می‌دهیم.

۹-۴ ساختار جدول صفحه

در این بخش، برخی تکنیک‌های رایج ساختار جدول صفحه، اعم از صفحه‌بندی سلسله‌مراتبی، جدول صفحات درهم و جدول صفحات وارونه[1] را مرور می‌کنیم.

۹-۴-۱ صفحه‌بندی سلسله‌مراتبی

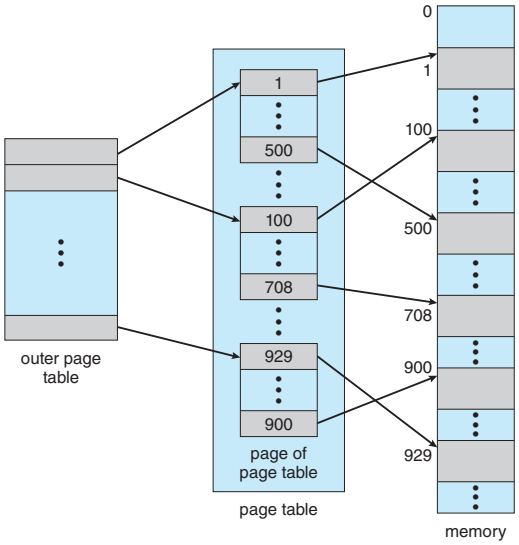
بیشتر سیستم‌های کامپیوتری مدرن، فضای آدرس منطقی بزرگی را پشتیبانی می‌کنند (2^{32} تا 2^{64} بایت). در چنین محیطی خود جدول صفحه می‌تواند بسیار بزرگ باشد. برای مثال، سیستمی با فضای آدرس منطقی ۳۲ بیتی را در نظر بگیرید. اگر اندازه‌ی صفحه در چنین سیستمی ۴ کیلوبایت (2^{12} بایت) باشد، در این صورت جدول صفحه می‌تواند بالای ۱ میلیون وارده ($2^{32}/2^{12} = 2^{20}$) داشته باشد. با فرض این که هر وارده ۴ بایت است؛ جدول صفحه به تنهایی، ۴ مگابایت از فضای آدرس فیزیکی هر پردازه را خواهد گرفت. روشن است که نمی‌خواهیم جدول صفحه را در حافظه‌ی اصلی به صورت پیوسته نگه بداریم. یک راه‌حل ساده برای این مسأله، تقسیم جدول صفحه به تکه‌های کوچک‌تر است. این تقسیم‌بندی را می‌توانیم به روش‌های متعددی انجام دهیم.

یک روش، استفاده از الگوریتم صفحه‌بندی دوسطحی است که در آن، خود جدول صفحه نیز صفحه‌بندی می‌شود (شکل ۹-۱۵). برای مثال، سیستم ۳۲ بیتی را باز هم با اندازه‌ی صفحات ۴ کیلوبایتی در نظر بگیرید. هر آدرس منطقی به یک شماره‌ی صفحه‌ی ۲۰ بیتی و افست صفحه‌ی ۱۲ بیتی تقسیم می‌شود. چون جدول صفحه را دوباره صفحه‌بندی می‌کنیم، شماره‌ی صفحه خود به یک شماره‌ی صفحه‌ی ۱۰ بیتی و یک افست صفحه‌ی ۱۰ بیتی تقسیم می‌شود. بنابراین، آدرس منطقی به صورت زیر خواهد بود:

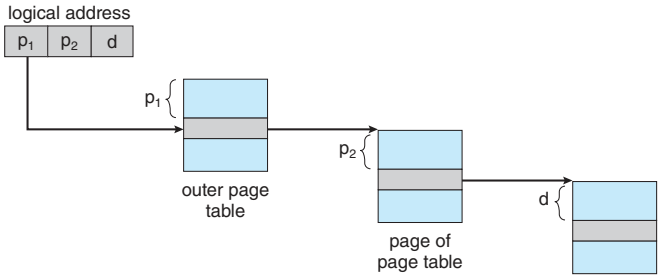
شماره‌ی صفحه		افست صفحه
p_1	p_2	d
۱۰	۱۰	۱۲

1. Inverted page tables

که در آن، p_1 ایندکسی به جدول صفحه‌ی بیرونی است و p_2 جابجایی در صفحه‌ای خواهد بود که از جدول صفحه‌ی بیرونی با ایندکس p_1 استخراج می‌شود. روش ترجمه‌ی آدرس این معماری در شکل ۹-۱۶ نشان داده می‌شود. چون ترجمه‌ی آدرس از جدول بیرونی شروع شده و به طرف داخل انجام می‌گیرد، این طرح به جدول صفحه‌ی **با نگاشت پیشرو**[1] نیز معروف است.



شکل ۹-۱۵ طرح جدول صفحه‌ی دوسطحی.



شکل ۹-۱۶ ترجمه‌ی آدرس برای یک معماری صفحه‌بندی ۳۲ بیتی دوسطحی.

طرح صفحه‌بندی دوسطحی در سیستمی با فضای آدرس منطقی ۶۴ بیتی باز هم نمی‌تواند مناسب باشد. برای روشن‌شدن این موضوع فرض کنید اندازه‌ی صفحه در چنین سیستمی برابر ۴ کیلوبایت (2^{12} بایت) باشد. در این حالت، جدول صفحه شامل 2^{52} وارده خواهد بود. اگر از طرح

1. Forward-mapped page table

صفحه‌بندی دوسطحی استفاده کنیم، در این صورت بزرگی جدول صفحات داخلی متعارف بوده و شامل 2^{10} وارده‌ی ۴ بایتی می‌باشند. آدرس‌ها به شکل زیر خواهند بود:

صفحه‌ی بیرونی	صفحه‌ی داخلی	افست
p_1	p_2	D
۴۲	۱۰	۱۲

جدول صفحه‌ی بیرونی، شامل 2^{42} وارده یا 2^{44} بایت است. روش بدیهی اجتناب از داشتن جدول صفحه‌ی بزرگ این چنینی آن است که جدول صفحه‌ی بیرونی به تکه‌های کوچک‌تر تقسیم شود. (این رویکرد نیز در برخی پردازنده‌های ۳۲ بیتی برای افزودن انعطاف و کارآمدی استفاده می‌شود.)

به شیوه‌های مختلفی می‌توانیم جدول صفحه‌ی بیرونی را تقسیم کنیم. برای مثال، می‌توانیم جدول صفحه‌ی بیرونی را صفحه‌بندی کنیم تا یک طرح صفحه‌بندی سه‌سطحی داشته باشیم. فرض کنید که جدول صفحه‌ی بیرونی دارای اندازه‌ی صفحات استانداردی باشد (2^{10} وارده‌ای یا 2^{12} بایتی). در اینجا، باز هم فضای آدرس ۶۴ بیتی هولناک است:

صفحه‌ی بیرونی دوم	صفحه‌ی بیرونی	صفحه‌ی داخلی	افست
p_1	p_2	p_3	d
۳۲	۱۰	۱۰	۱۲

اندازه‌ی جدول صفحه‌ی بیرونی هنوز 2^{34} بایت (۱۶ گیگابایت) است.

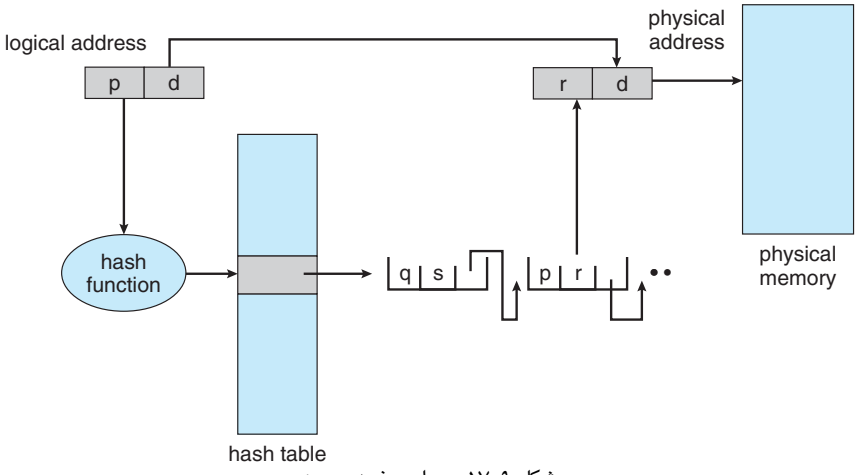
گام بعدی، طرح صفحه‌بندی چهارسطحی خواهد بود که در آن جدول صفحه‌ی بیرونی سطح دوم نیز خود صفحه‌بندی می‌شود و الی آخر. UltraSPARC ۶۴ بیتی برای ترجمه‌ی هر یک از آدرس‌های منطقی به هفت سطح صفحه‌بندی—با یک سری دسترسی گران حافظه—نیاز خواهد داشت. از این مثال می‌بینید که چرا جدول صفحات سلسله‌مراتبی در معماری‌های ۶۴ بیتی عموماً مناسب نیستند.

۹-۴-۲ جدول صفحات درهم

یک رویکردی که برای راه‌اندازی فضاهای آدرس بزرگ‌تر از ۳۲ بیت به کار می‌رود استفاده از **جدول صفحه‌ی درهم** با یک مقدار درهم است که شماره‌ی صفحه‌ی مجازی می‌باشد. هر وارده‌ی جدول درهم، یک لیست پیوندی (به منظور رفع برخوردها) از عناصر را شامل می‌شود که درهم آنها مکان یکسانی است. هر یک از این عناصر، سه فیلد دارند که عبارتند از: (۱) شماره‌ی صفحه‌ی مجازی، (۲) شماره‌ی قاب نگاشته شده و (۳) اشاره‌گری به عنصر بعدی لیست پیوندی.

الگوریتم به این صورت کار می‌کند: مقدار درهم شماره‌ی صفحه‌ی مجازی از روی آدرس مجازی برای جدول درهم محاسبه می‌شود. شماره‌ی صفحه‌ی مجازی با اولین فیلد نخستین عنصر لیست پیوندی مقایسه می‌گردد. اگر تطبیق داشت، قاب متناظر (فیلد ۲) جهت تشکیل آدرس

فیزیکی مطلوب مورد استفاده قرار می‌گیرد. اگر تطبیقی صورت نگیرد، واردهای بعدی لیست پیوندی برای یافتن شماره‌ی صفحه‌ی مجازی منطبق جستجو می‌شوند. این طرح در شکل ۹-۱۷ نشان داده می‌شود.



شکل ۹-۱۷ جدول صفحه‌ی درهم.

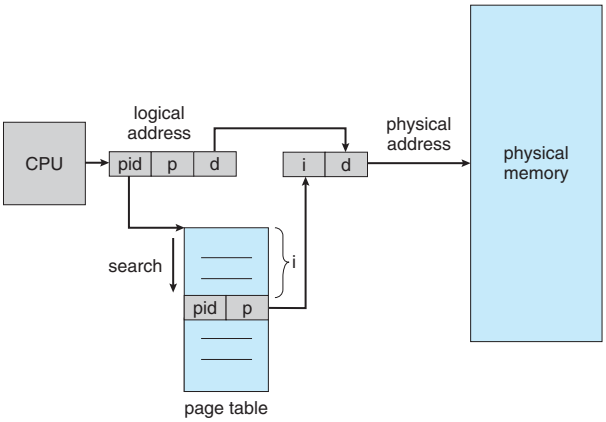
گونه‌ای از این طرح که تناسب بهتری با فضای آدرس ۶۴ بیتی دارد، پیشنهاد شده است. این طرح از **جدول صفحات خوشه‌ای**[1] استفاده می‌کند که مشابه جدول صفحات درهم هستند با این تفاوت که، هر یک از واردهای جدول درهم به جای یک صفحه‌ی منفرد به چندین صفحه (برای مثال، ۱۶ صفحه) رجوع می‌کنند. بنابراین، یک تک‌وارده از هر جدول صفحه می‌تواند نگاشت چندین قاب فیزیکی را ذخیره نماید. جدول صفحات خوشه‌ای مخصوصاً برای فضای آدرس‌های **پراکنده** مفید هستند که در آن مراجعات حافظه غیرهمجوار بوده و در کل فضای آدرس پخش شده‌اند.

۹-۴-۳ جدول صفحات وارونه

معمولاً، هر پردازه یک جدول صفحه‌ی مرتبط دارد. جدول صفحه برای هر صفحه‌ای که پردازه استفاده می‌کند یک وارده (یا یک شکاف برای هر آدرس مجازی، صرفنظر از اعتبار آدرس) دارد. این نمایش از جدول، طبیعی به نظر می‌رسد، چون مراجعات پردازه‌ها به صفحات از طریق آدرس‌های مجازی صفحات صورت می‌گیرند. در ادامه، سیستم‌عامل بایستی این مراجعات را به آدرس فیزیکی حافظه‌ی آنها ترجمه کند. از آنجایی که جدول مطابق آدرس مجازی مرتب می‌شود، سیستم‌عامل قادر است مکانی از جدول را که به آدرس فیزیکی وارده مربوط می‌شود محاسبه کرده و آن را مستقیماً استفاده نماید. یکی از معایب این روش آن است که هر یک از جدول صفحات می‌تواند تا میلیون‌ها وارده داشته باشد. این جدول‌ها حجم بسیار زیادی از حافظه‌ی فیزیکی را تنها به این دلیل مصرف می‌کنند که فقط رد چگونگی استفاده از سایر مکان‌های حافظه‌ی فیزیکی را نشان بدهند.

1. Clustered page tables

برای حل این مسأله می‌توانیم از یک **جدول صفحه‌ی وارونه** استفاده کنیم. هر جدول صفحه‌ی وارونه برای هر یک از صفحات واقعی حافظه (یا قاب‌ها) یک وارده دارد. هر وارده، آدرس مجازی صفحه‌ای را که در آن مکان واقعی ذخیره شده، به همراه اطلاعاتی در رابطه با پردازه‌ای که مالک آن صفحه می‌باشد در بر دارد. بنابراین، فقط یک جدول صفحه در سیستم وجود دارد که آن هم برای هر یک از صفحات حافظه‌ی فیزیکی تنها یک وارده دارد. شکل ۹-۱۸، عملیات جدول صفحه‌ی وارونه را نشان می‌دهد. این شکل را با شکل ۹-۸ که عملیات جدول صفحه‌ی معمولی را به تصویر می‌کشد، مقایسه کنید. جدول صفحات وارونه اغلب لازم دارند تا یک شناسه از فضای آدرس (بخش ۹-۳-۲) نیز با هر یک از وارده‌های جدول صفحه ذخیره شود، چرا که جدول معمولاً فضاهای آدرس متعددی را شامل است که به حافظه‌ی فیزیکی نگاشته شده‌اند. ذخیره‌ی شناسه‌ی فضای آدرس تضمین می‌کند که یک صفحه‌ی منطقی از پردازه‌ای خاص به قاب فیزیکی متناظر آن پردازه نگاشته می‌شود. مثال‌هایی از سیستم‌هایی که از جدول صفحات وارونه استفاده می‌کنند، عبارتند از: UltraSPARC ۶۴ بیتی و PowerPC.



شکل ۹-۱۸ جدول صفحه‌ی وارونه.

برای روشن‌شدن این روش، نسخه‌ی ساده شده‌ای از جدول صفحه‌ی وارونه‌ی استفاده شده در ***IBM RT*** را مطرح می‌کنیم. IBM اولین شرکت اصلی است که با شروع از IBM System 38 از جدول صفحات وارونه استفاده کرده و تا RS/6000 و پردازنده‌های IBM Power فعلی ادامه داشته است. در IBM RT، هر آدرس مجازی سیستم شامل سه‌تایی زیر است:

```
<process-id, page-number, offset>
```

هر یک از وارده‌های جدول صفحه‌ی وارونه به صورت یک زوج <process-id, page-number> است که در آن process-id نقش شناسه‌ی فضای آدرس را ایفا می‌کند. لحظه‌ای که یک مراجعه به حافظه اتفاق می‌افتد، بخشی از فضای آدرس که شامل <process-id, page-number> می‌باشد به زیرسیستم حافظه عرضه می‌شود. سپس، جدول صفحه‌ی وارونه برای یافتن یک انطباق جستجو می‌گردد. اگر تطابقی یافت شود (برای مثال، با وارده‌ی iام)، در این صورت آدرس

فیزیکی <i, offset> تولید می‌شود. اگر تطبیقی صورت نگیرد، در این صورت دسترسی صورت گرفته، یک اقدام دسترسی غیر قانونی آدرس قلمداد می‌شود.

اگرچه این طرح، حجم حافظه‌ای را که برای ذخیره‌سازی هر جدول صفحه لازم می‌شود کاهش می‌دهد، اما زمان لازم برای جستجوی جدول صفحه را در زمان رجوع به یک صفحه افزایش می‌دهد. چون جدول صفحه‌ی وارونه بر اساس آدرس فیزیکی مرتب می‌شود، حال آنکه جستجوی آن بر اساس آدرس‌های مجازی است، این امکان وجود دارد که کل جدول برای یافتن یک انطباق جستجو شود. این جستجو زمان زیادی را خواهد گرفت. برای تعدیل این مسأله از یک جدول درهم که در بخش ۹-۴-۲ مطرح شد، استفاده می‌کنیم تا جستجو را به یک وارده—یا حداکثر تعدادی معدود—محدود کنیم. البته، هر مرتبه دسترسی جدول درهم، یک مراجعه به حافظه اضافه می‌کند، لذا هر مراجعه به حافظه‌ی مجازی، حداقل دو بار خواندن واقعی حافظه—یکی برای وارده‌ی جدول درهم و دیگری برای جدول صفحه—را لازم خواهد داشت. (به یاد داشته باشید که پیش از مراجعه به جدول درهم، نخست TLB جستجو می‌شود که کارایی را تا اندازه‌ای بهبود می‌بخشد.)

یک موضوع جالب در رابطه با جدول صفحات وارونه مربوط به حافظه‌ی مشترک می‌باشد. در صفحه‌بندی معمولی، هر پردازه جدول صفحه‌ی خاص خود را دارد که اجازه می‌دهد تا آدرس‌های مجازی متعددی به آدرس فیزیکی یکسانی نگاشته شوند. این روش نمی‌تواند در جدول صفحات وارونه استفاده شود؛ چون برای هر یک از صفحات فیزیکی، فقط یک وارده برای صفحه‌ی مجازی وجود دارد، یک صفحه‌ی فیزیکی نمی‌تواند دو (یا چند) آدرس مجازی مشترک داشته باشد. بنابراین، در جدول صفحات وارونه در یک زمان مشخص، فقط یک نگاشت از یک آدرس مجازی به آدرس فیزیکی مشترک می‌تواند اتفاق بیفتد. یک ارجاع توسط پردازه‌ی دیگری که حافظه را به اشتراک گرفته به یک خطای صفحه منجر شده و نگاشت را با یک آدرس مجازی متفاوتی جایگزین خواهد کرد.

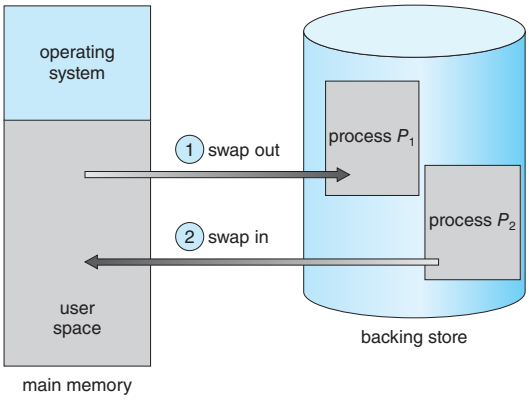
۹-۴-۴ اوراکل‌اسپارک سولاریس

به عنوان مثال آخری، یک پردازنده‌ی ۶۴ بیتی مدرن و سیستم‌عاملی که به صورت سفت و سخت تجمیع شده‌اند تا یک حافظه‌ی مجازی با سربار کم فراهم کنند را در نظر بگیرید. **سولاریس** با اجرا بر روی پردازنده‌ی اسپارک، یک سیستم‌عامل ۶۴ بیتی کامل است و بر آن است تا با حفظ سطوح متعددی از جدول صفحات، مسأله‌ی حافظه‌ی مجازی را بدون استفاده از کل حافظه‌ی فیزیکی حل کند. رویکرد آن کمی پیچیده است اما مسأله را با استفاده از جدول صفحات درهم به صورت کارآمد حل می‌کند. دو جدول درهم–یکی برای هسته و یکی هم برای تمامی پردازه‌های کاربر–وجود دارند. هر یک، آدرس‌های حافظه را از حافظه‌ی مجازی به فیزیکی نگاشت می‌دهند. هر وارده‌ی جدول درهم، معرف یک ناحیه‌ی همجوار از حافظه‌ی مجازی نگاشته شده است که کارآمدتر از داشتن یک وارده‌ی مجزای جدول برای هر صفحه است. هر وارده یک آدرس مبنا و یک محدوده دارد که نشان از تعداد صفحات وارده است.

ترجمه‌ی مجازی-به-فیزیکی، زیادی به طول خواهد انجامید در صورتی که هر آدرس مستلزم جستجوی جدول درهم باشد، لذا پردازنده یک TLB پیاده‌سازی می‌کند که وارده‌های جدول ترجمه[1] (TTEهایی) را برای جستجوهای پرسرعت سخت‌افزاری نگه می‌دارد. کَشی از این TTEها در بافر ذخیره‌ی ترجمه[2] (TSB) مقیم است که یک وارده برای هر صفحه‌ی اخیراً دستیابی شده لحاظ می‌کند. موقعی که یک رجوع به آدرس مجازی صورت می‌گیرد، سخت‌افزار، TLB را برای یک ترجمه جستجو می‌کند. اگر چیزی پیدا نشود، سخت‌افزار برای جستجوی TTE متناظر با آدرس مجازی مسبب مراجعه، در TSB داخل حافظه سرکی می‌کشد[3]. این کارکرد **سَرَک TLB** در بسیاری از پردازنده‌های مدرن یافت می‌شود. اگر تطابقی در TSB پیدا شود، پردازنده وارده‌ی TSB را به TLB کپی کرده و ترجمه‌ی حافظه کامل می‌شود. اگر هیچ تطبیقی در TSB پیدا نشود، هسته وقفه می‌گیرد تا جدول درهم را جستجو کند. سپس، هسته یک TTE از جدول درهم مناسبی ایجاد کرده و آن را در TSB برای بارگذاری خودکار به داخل TLB توسط واحد مدیریت حافظه‌ی پردازنده ذخیره می‌نماید. در نهایت، راه‌انداز وقفه، کنترل را به MMU برمی‌گرداند که ترجمه‌ی آدرس را تکمیل کرده و بایت یا کلمه‌ی درخواستی را از حافظه‌ی اصلی بازیابی می‌کند.

۹-۵ مبادله

دستورالعمل‌های پردازه و داده‌هایی که بر روی آن‌ها عملیات دارند بایستی برای اجرا در حافظه قرار بگیرند. با این وجود، یک پردازه یا بخشی از آن می‌تواند موقتاً به بیرون حافظه به **ذخیره‌ی پشتیبان**[4] **مبادله شود**[5] و در ادامه، برای تداوم اجرا به داخل حافظه آورده شود (شکل ۹-۱۹). مبادله، این امکان را به وجود می‌آورد که فضای آدرس فیزیکی تمامی پردازه‌ها در مجموع از حافظه‌ی فیزیکی واقعی سیستم فراتر رود، از این رو درجه‌ی چندبرنامگی در یک سیستم بالا می‌رود.



شکل ۹-۱۹ مبادله‌ی استاندارد دو پردازه با استفاده از یک دیسک به عنوان ذخیره‌ی پشتیبان.

1. Translation table entries
2. Translation storage entry
3. Walks through
4. Backing store
5. Swapped